

Experiment 3

Software Design Principles

Identify stakeholders and perform requirement elicitation for the selected software project using suitable techniques such as interviews, questionnaires, observation, or user personas; prepare user stories and the Requirement Gathering Report.

Aim

To identify stakeholders and perform requirement elicitation for the ExamEnv system using suitable elicitation techniques, prepare user stories, and develop a Requirement Gathering Report.

Objectives

- Identify key stakeholders of the ExamEnv system.
- Apply requirement elicitation techniques.
- Gather functional and non-functional requirements.
- Prepare user stories with acceptance criteria.
- Document the Requirement Gathering Report.

Introduction

Requirement elicitation is the process of discovering stakeholder needs and expectations. The quality of software depends on how accurately user requirements are captured. Common elicitation techniques include interviews, questionnaires, observation, workshops, and user personas. For ExamEnv, interviews, questionnaires, observation, and user personas were used to understand both teacher-side and student-side workflows.

Selected Project

Project Title: ExamEnv Infrastructure-as-Code for Exams

Domain: Educational Technology (EdTech) and DevOps Automation

Stakeholder Identification

A. Human / Organizational Stakeholders

Stakeholder	Role	Key Requirements
Teacher / Course Instructor	Uploads exam requirements, reviews/edits generated YAML, monitors student progress	Accurate parsing, easy review/editing, real-time visibility into student readiness
Student / Exam Candidate	Provisions and verifies exam environment via CLI agent	Fast, one-command setup with clear preview and reliable verification
Teaching Assistant / Lab Support	Helps students during labs and exams	Dashboard visibility, quick error diagnosis, student status filters

System Administrator / DevOps Engineer	Maintains infrastructure (Celery/Redis, PostgreSQL, WebSocket service)	Monitoring, logging, easy deployment and scaling
Stakeholder	Role	Key Requirements
Exam Cell / University IT (optional)	Approves exam environment standards	Security, audit logs, compliance

B. Internal / System Components

Component	Role	Key Requirement
Backend Server	Stores files, orchestrates async parsing jobs, serves APIs, tracks status	Reliable async processing, consistent API contracts, accurate status tracking
ML / Parser Engine	Extracts structured requirements from uploaded PDFs and converts them to YAML	Well-formatted input documents, clear extraction schema, template library
Student Agent (CLI)	Fetches YAML, performs dry-run, builds/runs Docker container, runs verification	Stable Docker environment, correct YAML spec, network access to backend
Docker Engine	Runs isolated containers	Correct port and volume mapping
PostgreSQL + Redis/Celery	Database and asynchronous task queue	Data integrity, reliable job processing

Requirement Elicitation Techniques

Technique	Participants / Source	Purpose	Outcome
Interview	3 teachers, 5 students	Discuss exam-setup pain points	Detailed functional requirements for upload, parsing and provisioning
Questionnaire	30 students across different courses	Collect feedback from multiple students	Common environment issues and feature preferences
Observation	Lab/exam setup session	Observe students manually setting up environments	Time wasted and typical failure points in manual setup
User Persona	Teacher, Student, Teaching Assistant	Model representative users	User-centred requirements for dashboard and CLI agent

Sample Requirement Gathering

Interview Findings

- Teachers want to upload existing requirement documents (PDF) rather than writing YAML by hand.
- Teachers need to review and correct the AI-generated YAML before it is published to students.
- Students want a clear preview of what will be installed before anything changes on their machine.
- Environment setup should complete within a few minutes on a typical laptop.
- Teaching Assistants want to quickly identify which students are stuck and why.

Observation Findings

- Students often struggle most with version mismatches (e.g. wrong Python or package version) and port conflicts.
- Teachers spend a large amount of time answering the same setup questions individually during labs.
- Manual setup can take 30–45 minutes per student and often fails midway.

Questionnaire Findings

- Students expect setup to work identically across Windows, macOS and Linux.
- Reliability and clear error messages are rated as the highest priorities, followed by setup speed.
- Most students prefer a one-command setup with a dry-run preview before changes are applied.

User Personas

Persona	Age	Tech Skill	Goal	Pain Point	Needs
Teacher – Course Instructor	38	Medium	Publish a correct, ready-to-use exam environment spec quickly	AI-generated YAML sometimes misses edge-case requirements	Easy review/edit, publish, real-time student status
Student – Exam Candidate	20	Medium	Get a working environment with one command before the exam starts	Manual setup steps in the past have eaten into exam time	One-command setup, dry-run preview, reliable verification
Teaching Assistant – Lab Support	24	High	Quickly see which students are stuck and why	No visibility into individual student failures without the dashboard	Dashboard, error logs, status filters

User Stories

ID	User Story	Acceptance Criteria	Priority	Linked FR
US1	As a teacher, I want to upload my exam requirements as a PDF so that I don't have to write YAML manually.	Given a logged-in teacher, when they upload a valid PDF, then the system stores the file and starts asynchronous parsing. The dashboard shows "Processing". Invalid files show a clear error message. The teacher can see when parsing is complete.	High	FR1, FR2
US2	As a teacher, I want to review and edit the AI-generated YAML before publishing so that I can fix any parsing mistakes.	Given generated YAML, when the teacher edits fields and clicks "Publish", then the edited YAML becomes the published version for students. Validation errors are shown clearly before publishing.	High	FR3
US3	As a student, I want to run a single CLI command to provision my exam environment so that I don't waste time on manual setup.	Given the CLI is installed and the student is authenticated, when they run <code>examenv init <exam-id></code> , then the agent fetches the published YAML and displays the exam name. If no exam is available, a clear error is shown.	High	FR4, FR6

US4	As a student, I want to see a dry-run preview of what will be installed before confirming so that I know what changes will be made.	Given the student has fetched the published YAML, when they run dry-run, the agent lists packages, versions, ports, volumes, and setup commands. No changes are made until the student types "Yes". If the student types "No", the agent exits without changes.	High	FR5
US5	As a student, I want my environment to be automatically verified after setup so that I know it is ready for the exam.	Given setup completes, when verification runs, then it checks Python version, installed packages, imports, and ports. The agent reports "Ready" or "Failed" with logs. "Ready" is shown only if all checks pass.	High	FR7, FR8
US6	As a teacher, I want to see each student's status (Ready/Failed/Not Started) in real time so that I can help students who are stuck.	Given the teacher dashboard is open, when a student's status changes, it updates within 2 seconds to Ready, Failed, or Not Started. Clicking a Failed student shows the exact error logs.	High	FR9, FR10

Functional Requirements

ID	Requirement
FR1	Teacher shall upload exam requirement PDFs.
FR2	System shall asynchronously parse PDFs into YAML using Celery + Redis.
FR3	Teacher shall review, edit, and publish generated YAML.
FR4	Student Agent shall fetch published YAML for the exam.
FR5	Agent shall show a dry-run preview before applying changes.
FR6	Agent shall build and run a Docker container from the YAML spec.
FR7	Verification script shall check dependencies inside the container.
FR8	Agent shall report Ready/Failed status to the backend.
FR9	Teacher dashboard shall show real-time student status.
FR10	Teacher shall view error logs for failed students.

Non-Functional Requirements

ID	Requirement
NFR1	Environment setup should complete within a few minutes on a typical laptop.
NFR2	System shall work on Windows, macOS, and Linux.
NFR3	Communication and authentication must be secure.
NFR4	Async parsing jobs should be reliable and retry on failure.
NFR5	Dashboard should update within 2 seconds via WebSockets.
NFR6	Error messages must be clear and actionable.

NFR7	System should scale to at least 100 concurrent students.
------	--

Requirement Gathering Report

The elicitation process identified PDF-to-YAML parsing, teacher review and publishing, one-command student provisioning, dry-run preview, in-container verification, and real-time teacher monitoring as the primary requirements for ExamEnv.

Stakeholder feedback emphasised speed of setup, accuracy of the AI-generated specification, cross-platform reliability, and clear visibility into student readiness during labs and exams.

Using multiple elicitation techniques interviews with teachers and students, observation of current manual setup pain points, and questionnaires improved the completeness of the requirements and reduced ambiguity between the teacher-side and student-side workflows.

The functional requirements (FR1–FR10) and non-functional requirements (NFR1–NFR7) were documented, and user stories with specific acceptance criteria were prepared.

Conclusion

The Requirement Gathering Report for ExamEnv Infrastructure-as-Code for Exams was successfully completed. Stakeholders were identified, requirements were elicited using suitable techniques, and user stories were prepared.

Result

Stakeholders were identified, requirements were elicited using suitable techniques, user stories were prepared, and the Requirement Gathering Report for the ExamEnv Infrastructure-as-Code for Exams project was successfully completed.